UNIVERSITÀ DEGLI STUDI DI PADOVA

DIPARTIMENTO DI MATEMATICA

LAUREA TRIENNALE IN INFORMATICA

PROGETTO DI BASI DI DATI

Basi di Dati di un Sistema Informativo per la Gestione Logistica e delle Spedizioni

Zakaria Laoud - Edis Hodja

Indice

1 Abstract **2**

2 Analisi dei Requisiti **2**

3 Progettazione Concettuale **5**

4 Progettazione Logica **6**
4.1 Analisi delle Ridondanze 8
4.1.1 Con ridondanza 8
4.1.2 Senza ridondanza 9
4.2 Eliminazioni delle Generalizzazioni 9
4.3 Schema Relazionale 10

5 Implementazione in PostgreSQL e Definizione delle Query **11**
5.1 Definizione delle Query 11
5.2 Creazione degli indici 14

6 Applicazione Software **14**

1 Abstract

Il seguente progetto permette la gestione e supervisione di un sistema di spedizioni, e l'amministrazione di tutti gli aspetti che accomunano generalmente un magazzino predisposto alla consegna della posta.

Nel dettaglio, il sistema si sviluppa andando a considerare vari aspetti che riguardano il tracciamento dei vari pacchi di un determinato ordine, la gestione dei dipendenti nei vari magazzini, suddivisi in base al ruolo, e l'amministrazione dei vari mezzi messi a disposizione per le consegne.

Una composizione più vasta si espande creando anche una vera e propria gerarchia all'interno dei vari ruoli di lavoro, in modo da esser il più fedele possibile a un vero contesto professionale.

Questa progettazione punta alla realizzazione di un modello di controllo dei dati funzionale, monitorato ed esaustivo, che permetta sia al cliente che al gestore delle spedizioni di disporre di un controllo totale dei vari ordini e relativi pacchi.

2 Analisi dei Requisiti

I requisiti sulla quale la basi di dati si basa sono:

Cliente - Nell'entità cliente saranno memorizzati sia i mittenti che i destinatari dei vari ordini, e tra le informazioni che vengono registrate troviamo:

- Codice_cliente che identifica univocamente ogni cliente
- Nome e Cognome
- Telefono
- Email (se fornitaci)
- Indirizzo, suddiviso in: riga_indirizzo1 , riga_indirizzo2 (se fornitoci), Città, CAP e Paese

Ordine - Nell'entità ordine vengono memorizzati tutti gli ordini mediante alcune informazioni basilari tipo:

- Il numero_ordine che identifica univocamente l'ordine
- La data nella quale un ordine viene inserito nel sistema

Tracciamento - I tracciamenti comprendono:

- Codice_tracciamento univoco
- Data-ora che rappresenta l'ultima variazione di stato

- Stato che può essere 'In attesa', 'In magazzino', 'In consegna' oppure 'Consegnato'

I codici di tracciamento sono relativi all'identificazione di ciascun pacco, mentre il numero dell'ordine identifica, per intero, un insieme di prodotti che nei sistemi di gestione delle spedizioni risulta utile per il servizio clienti.

Pacco - I pacchi, oltre alle informazioni relative al tracciamento, sono identificati attraverso:

- Codice di tracciamento (fornito dalla relazione *riferisce*).
- Tipo_contenuto che distingue la categoria di appartenza dei prodotti all'interno del collo(se fornitoci)
- Peso e Dimensioni.

Magazzino - I magazzini sono strutture adibite alla gestione dei vari pacchi, e rappresentano un punto di riferimento per l'amministrazione delle spedizioni con i seguenti attributi:

- Codice identificativo di ciascun magazzino di riferimento
- Città di ubicazione
- Nome
- Capacità massima a livello di pacchi che possono essere ospitati
- Numero di pacchi gestiti al momento

Uno stesso magazzino può occuparsi di più pacchi, rispettando la capienza massima che gli è stata attribuita. Ciò ovviamente escludendo tutti i pacchi che risultano già consegnati.

Dipendente - L'entità dipendente è composta da tutti coloro che lavorano per l'azienda di spedizioni, ognuno dei quali è rappresentato da:

- Codice fiscale univoco per ogni dipendente
- Nome e Cognome
- Telefono
- Email
- Responsabile (l'unico dipendente a non avere un responsabile è il proprietario dell'azienda)
- Ruolo del dipendente

- Abilitazione in riferimento al ruolo di magazziniere (se il dipendete in questione è un magazziniere)
- Ufficio in riferimento al ruolo di segretario (se il dipendente in questione è un segretario)

I dipendenti si suddividono per categoria:

- **Segretario** Oltre alle informazioni generali, per i segretari viene registrato l'ufficio presso il quale operano
- **Magazziniere** Oltre alle informazioni generali, per i magazzinieri viene registrata l'abilitazione (Carrellista, Ricezionista, Picker)
- **Corriere** Oltre alle informazioni generali, per i corrieri registrano anche la patente acquisita che ci permette di comprendere la tipologia di veicolo da assegnargli

Veicolo - I veicoli sono i mezzi tramite i quali avvengono le varie consegne e sono caratterizzati dai seguenti attributi:

- Targa
- Capacità cioè il numero di pacchi massimo che possono essere trasportati
- Patente richiesta
- Ultima manutenzione

Oltre alle informazioni generali, per i veicoli si generalizza in **Bici con rimorchio, Furgone e Motorino**

L'entità **Guida** tra Corriere e Veicolo, oltre ad essere caratterizzata da una data di utilizzo, ci consente di assegnare, in giorni alterni, i vari mezzi ai corrieri.

Consegna - Le consegne sono caratterizzate da:

- Esito della consegna
- Data nella quale è avvenuto il recapito o il tentativo di consegna
- Codice di tracciamento univoco (ricavato dalla relazione *riguardante*)

Opera - Rappresenta la relazione tra l'entità Corriere e Consegna e viene identificata tramite la combinazione delle chiavi esterne:

- Data di identificazione nella quale è avvenuto il recapito.
- Codice del corriere di riferimento che ha preso in carico l'operazione.

- Codice di tracciamento univoco.

Un pacco può esser soggetto ad un tentativo di consegna da corrieri diversi in giorni diversi.

Inoltre, prevedendo che ogni corriere possa esser affiancato, durante il regolare turno di lavoro, da un suo collega, una consegna può esser operata da più corrieri (corriere in stage oppure consegne di materiale pesante per esser spostato da una sola persona).

3 Progettazione Concettuale

Nella seguente figura viene mostrato il diagramma Entità-Relazione che riassume i requisiti descritti nella sezione precedente.

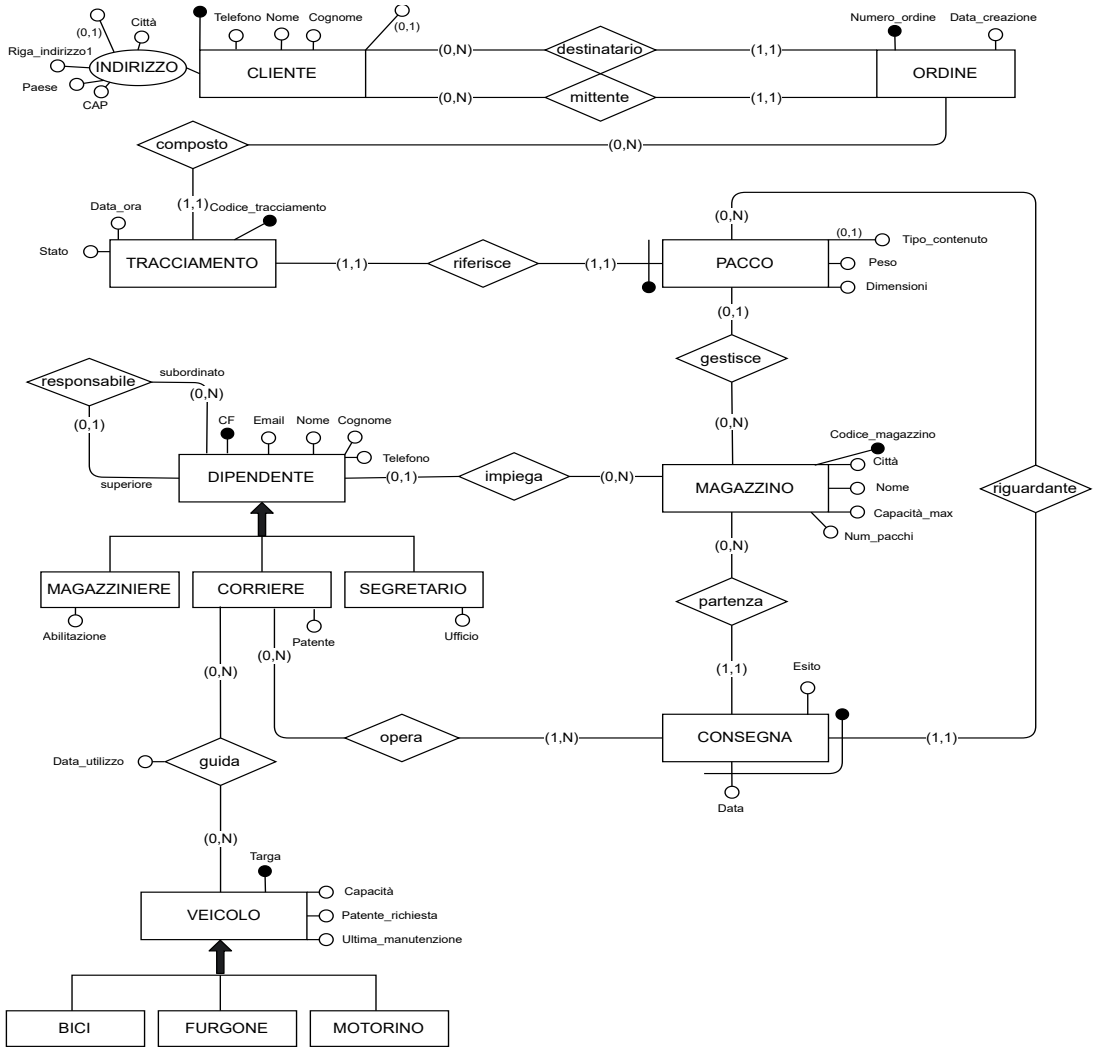

Figura 1: Diagramma Entità-Relazione.

Il sistema gestisce varie operazioni riguardanti un servizio di logistica e spedizioni che coinvolge clienti, ordini, pacchi, magazzini e dipendenti, tra i quali l'attenzione ricade sul corriere.

Un cliente è in grado di effettuare zero o più ordini, ognuno dei quali può essere composto da più pacchi, ognuno caratterizzato da un codice di tracciamento. Vengono tenute in conto informazioni aggiuntive, tra cui lo stato e l'ultima data in cui esso è stato variato. I pacchi comprendono diverse tipologie di prodotti, categorizzati in base al contenuto, di cui si tiene traccia sia del peso che delle dimensioni.

I magazzini sono delle strutture in cui i pacchi sono temporaneamente collocati e gestiti. Ciascun magazzino è identificato da un codice di riferimento con generalità aggiuntive riguardanti la città, il nome, la capacità massima di pacchi e il numero di pacchi in storage.

Il personale presente nel magazzino è composto da vari dipendenti supervisionati da un responsabile. I dipendenti vengono categorizzati in magazziniere, con abilitazione annessa, segretario, con ufficio assegnato e corriere con patente acquisita. Al corriere viene data la possibilità, ogni giorno, di utilizzare un veicolo diverso rispetto al giorno precedente. I mezzi di trasporto si generalizzano in bici con rimorchio, furgone e motorino, ciascuno con determinate informazioni che lo differenziano.

Le consegne sono operate dai corrieri, i quali ne hanno la responsabilità e sono identificate dalla data e dal codice di tracciamento riferito al pacco.

4 Progettazione Logica

Questa fase viene dedicata alla trasformazione dello schema concettuale appena presentato in uno schema logico facendo leva su analisi delle ridondanze, eliminazione delle generalizzazioni, accorpamento di entità e relazioni seguita da un'attenta scelta degli identificatori. In fine viene costruito lo schema ER ristrutturato.

Entità	**Descrizione**	**Attributi**	**Identificatore**
CLIENTE	Soggetto che può inviare e ricevere ordini	Codice_cliente, Nome, Cognome, Email, Telefono, Riga_indirizzo1, Riga_indirizzo2, CAP, Città, Paese	Codice_cliente
ORDINE	Spedizione di un insieme di colli	Numero_ordine, Data_creazione	Numero_ordine
TRACCIAMENTO	Codice di tracking dei pacchi	Codice_tracciamento, Data_ora, Stato	Codice_tracciamento
PACCO	Merce ordinata	Tipo_contenuto, Peso, Dimensioni	Relazione "riferisce"
MAGAZZINO	Warehouse dei vari pacchi	Codice_Magazzino, Città, Nome, Capacita_Max, Num_Pacchi	Codice_Magazzino

Entità	Descrizione	Attributi	Identificatore
DIPENDENTE	Impiegato del magazzino	Codice_Fiscale, Nome, Cognome, Email, Telefono, Responsabile	Codice_Fiscale
MAGAZZINIERE	Operaio	Abilitazione	
CORRIERE	Addetto alle consegne	Patente	
SEGRETARIO	Impiegato negli uffici	Ufficio	
VEICOLO	Mezzo di trasporto	Targa, Capacità, Patente_richiesta, Ultima_manutenzione	Targa
FURGONE	Patente richiesta: B		
BICI CON RIMORCHIO	Patente richiesta: AM		
MOTORINO	Patente richiesta: AM		
CONSEGNA	Operazione verso destinatario	Esito, Data	Relazione "riguardante" e Data

Tabella 1: Entità

Relazione	Descrizione	Componente	Attributo
Destinatario	Colui che riceve i pacchi	Ordine-Cliente	
Mittente	Colui che spedisce i pacchi	Cliente-Ordine	
Composto	Articolazione degli ordini	Ordine-Tracciamento	
Riferisce	Assegnazione del codice al pacco	Tracciamento-Pacco	Codice-tracciamento
Gestisce	Gestione del pacco presso magazzino	Pacco-Magazzino	
Impiega	Magazzino composto da dipendenti	Magazzino-Dipendente	
Responsabile	Superiore	Dipendente-Dipendente	CodiceFiscale
Guida	Mezzo guidato dal corriere	Corriere-Veicolo	Data-utilizzo
Opera	Consegna operata dal corriere	Corriere-Consegna	
Partenza	Magazzino dal quale parte la consegna	Magazzino-Consegna	
Riguardante	Consegna inerente ad un pacco	Consegna-Pacco	Codice-tracciamento

Tabella 2: Relazione

4.1 Analisi delle Ridondanze

L'attributo NUM_PACCHI, all'interno dell'entità MAGAZZINO, memorizza il numero di pacchi che sono attualmente in gestione all'interno di ogni singolo magazzino. Ovviamente tale numero può essere ricalcolata ogni volta che se ne ha bisogno, mediante una semplice operazione di JOIN attraverso la relazione GESTISCE.

Tra le possibili operazioni che un operatore può eseguire troviamo:

- **Operazione 2 (4 volte/giorno)**: un segretario visualizza i pacchi in gestione nei prossimi giorni (programmare i turni di lavoro)
- **Operazione 1 (1000 volte/giorno)**: viene assegnato un pacco in gestione ad un magazzino

Durante l'analisi dei costi assumeremo che un operazione di scrittura abbia costo doppio rispetto a una semplice operazione di lettura, e che la dimensione media di dati contenuta nelle singole entità sia:

Concetto	Costrutto	Dimensione
Magazzino	Entità	10
Gestisce	Relazione	30000
Pacco	Entità	30000

4.1.1 Con ridondanza

- Operazione 1:

Concetto	Costrutto	Accessi	Tipo	Ripetizione
Pacco	Entità	1	S	$\times 1000$
Gestisce	Relazione	1	S	$\times 1000$
Magazzino	Entità	1	L	$\times 1000$
Magazzino	Entità	1	S	$\times 1000$

- Operazione 2:

Concetto	Costrutto	Accessi	Tipo	Ripetizione
Magazzino	Entità	1	L	$\times 4$

Quindi volendo calcolare il costo, a livello di accessi, abbiamo:

$$\text{Costo Totale} = 1000 \times 3 \times 2 + 1000 + 4 = \mathbf{7004}$$

4.1.2 Senza ridondanza

- Operazione 1:

Concetto	Costrutto	Accessi	Tipo	Ripetizione
Pacco	Entità	1	S	×1000
Gestisce	Relazione	1	S	×1000

- Operazione 2:

Concetto	Costrutto	Accessi	Tipo	Ripetizione
Magazzino	Entità	1	L	×4
Gestisce	Relazione	3000	S	×4

In questo caso il numero di accessi è:

$$\text{Costo Totale} = 1000 \times 2 \times 2 + 3000 \times 4 + 4 = \mathbf{16004}$$

Dalla precedente analisi dei costi risulta quindi molto più utile, a livello computazionale, mantenere il campo ridondante.
Inoltre, dato che da ogni consegna possiamo risalire al corriere che l'ha operata, possiamo eliminare la relazione *partenza* senza perdita di dati.

4.2 Eliminazioni delle Generalizzazioni

Il modello relazionale non supporta direttamente concetti di ereditarierà o di generalizzazione, quindi esse vanno sostituite con entità o relazioni.

DIPENDENTE - La generalizzazione totale di *DIPENDENTE* viene rimossa andando ad applicare una delle principali possibilità viste a lezione, ovvero la soluzione ibrida che includeva l'accorpamento di alcune figlie nel padre e la sostituzione delle generalizzazioni con relazioni delle figlie più complesse. La prima operazione prevede che i ruoli dei vari dipendenti vengono gestiti direttamente all'entità DIPENDENTE, aggiungendo gli attributi di *SEGRETARIO* e *MAGAZZINIERE*. Invece, il ruolo di *CORRIERE* viene mantenuto come entità, aggiungendo la relazione *DATI_CORRIERE*, garantendo così che ogni corriere, oltre ad essere visto come un dipendente, abbia collegati a sé anche i dati relativi al ruolo ricoperto.

VEICOLO - Invece la generalizzazione *VEICOLO*, con le relative tipologie: *BICI, FURGONE, MOTORINO*, sono state accorpate come attributo *tipo* nella relazione padre.

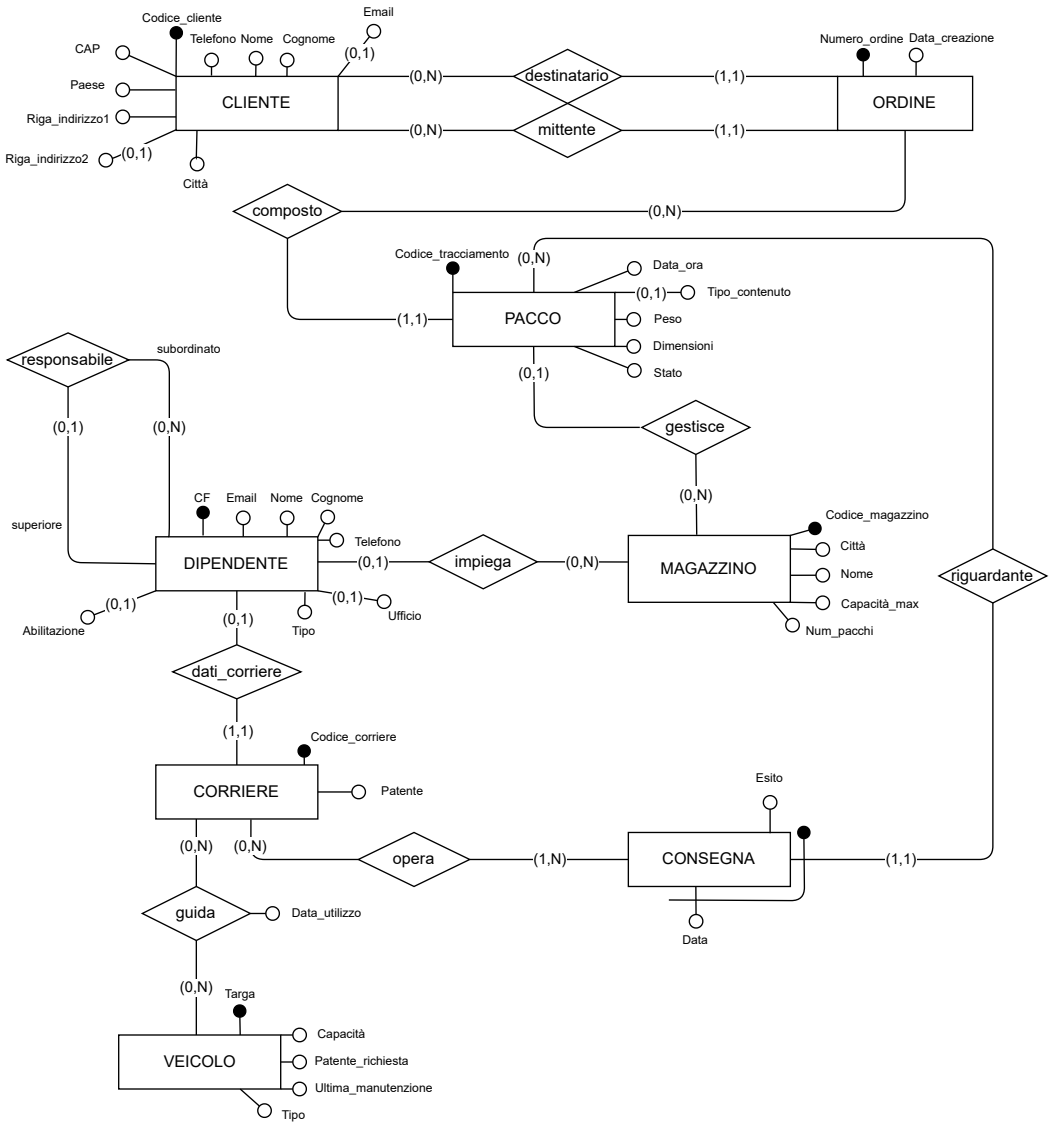

Figura 2: Diagramma Entità-Relazione ristrutturato.

4.3 Schema Relazionale

La figura sovrastante rappresenta lo schema relazionale ristrutturato, ottenuto applicando una serie di trasformazioni volte a migliorare l'efficienza e la chiarezza. Lo schema logico seguente riporta un'analisi più approfondita:

- **Cliente** (Codice, Nome, Cognome, Telefono, Email*, Riga_indirizzo1, Riga_indirizzo2*, Città, CAP, Paese)
- **Ordine** (Numero_ordine, Data_creazione, Cliente Mittente, Cliente Destinatario)
  - Ordine.Mittente → Mittente.Codice
  - Ordine.Destinatario → Destinatario.Codice

- **Pacco** (Codice_tracciamento, Data_ora, Tipo_contenuto*, Peso, Dimensioni, Stato, Ordine Numero_ordine, Magazzino Codice_magazzino)
  - Pacco.Numero_ordine → Ordine.Numero_ordine
  - Pacco.Codice_magazzino → Magazzino.Codice_magazzino
- **Magazzino** (Codice_magazzino, Città, Nome, Capacità_max, Num_pacchi)
- **Dipendente** (CF, Nome, Cognome, Telefono, Email, Tipo, Ufficio*, Abilitazione*, Dipendente Responsabile*, Magazzino Codice_magazzino*, Corriere Codice_corriere*)
  - Dipendente.CF → Dipendente.CF
  - Dipendente.Codice_magazzino → Magazzino.Codice_magazzino
  - Dipendente.Codice_corriere → Corriere.Codice_corriere
- **Corriere** (Codice_corriere, Patente, CF)
- **Guida** (Corriere Codice_corriere, Veicolo Targa, Data_utilizzo)
  - Guida.Codice_corriere → Corriere.Codice_corriere
  - Guida.Targa → Veicolo.Targa
- **Veicolo** (Targa, Capacità, Patente_richiesta, Ultima_manutenzione, Tipo)
- **Opera** (Corriere Codice_corriere, Data, Pacco Codice_tracciamento)
  - Opera.Codice_corriere → Corriere.Codice_corriere
  - Opera.Codice_tracciamento → Pacco.Codice_corriere
- **Consegna** (Pacco Codice_tracciamento, Data, Esito)
  - Consegna.Codice_tracciamento → Pacco.Codice_tracciamento

5 Implementazione in PostgreSQL e Definizione delle Query

Il file spedizioniere.sql contiene il codice SQL per la formazione e l'arricchimento delle tabelle del database.

5.1 Definizione delle Query

Di seguito vengono presentate e descritte le query con i relativi output generati e viene motivato l'utilizzo dell'indice proposto.

Query 1 Dato in ingresso il codice di un magazzino **N**, tale query mostra la quantita e il peso medio dei i vari pacchi attualmente in gestione in tale magazzino, raggruppati per tipo di contenuto.

```
SELECT tipo_contenuto, COUNT(*) as quantita, ROUND(AVG(p.peso),2) as media_peso
FROM pacco p, magazzino m
WHERE p.codice_magazzino = m.codice_magazzino
AND p.codice_magazzino = N
AND (p.stato = 'In magazzino' OR p.stato = 'In attesa')
GROUP BY tipo_contenuto
ORDER BY tipo_contenuto
```

Di seguito un'estratto dell'output con input **N=1**:

tipo_contenuto character varying (50)	quantita bigint	media_peso numeric
Abbigliamento	1	820.00
Accessori	1	650.00
Elettronica	2	1275.00
Giocattoli	5	2500.00
Libri	6	1271.67
[null]	2	2500.00

Query 2 Dati in ingresso una città **C** e un anno **A**, la seguente query da in output il numero di pacchi degli ordini che sono stati creati da tale città in tale anno.

```
SELECT COUNT(*) FROM pacco p
JOIN ordine o ON p.numero_ordine = o.numero_ordine
JOIN cliente c ON c.codice_cliente = o.mittente
WHERE c.citta = C AND EXTRACT(YEAR FROM o.data_creazione) = A
```

Di seguito un'estratto dell'output con input **C=Roma** e **A=2024**:

count bigint
20

Query 3 La seguente query da in output tutti i veicoli della compagna con associata la data, il codice del corriere e la città di ultimo utilizzo

```
SELECT v.tipo, r.targa, g.codice_corriere, r.data_ultimo_utilizzo, m.citta
FROM guida g RIGHT JOIN (
    SELECT v.targa, MAX(data_utilizzo) AS data_ultimo_utilizzo FROM veicolo v
    LEFT JOIN guida g ON g.targa = v.targa GROUP BY v.targa ) r
ON g.targa = r.targa AND g.data_utilizzo = r.data_ultimo_utilizzo
LEFT JOIN veicolo v ON v.targa = r.targa
LEFT JOIN dipendente d ON d.codice_corriere = g.codice_corriere
LEFT JOIN magazzino m ON m.codice_magazzino = d.codice_magazzino
ORDER BY data_ultimo_utilizzo, g.codice_corriere
```

Estratto dell'output:

tipo character varying (50)	targa character varying (7)	codice_corriere character varying (4)	data_ultimo_utilizzo date	citta character varying (50)
Bici con rimorchio	BC23456	A567	2024-02-15	Roma
Moto	MT12345	O210	2024-04-05	Firenze
Moto	MT23456	A789	2024-07-08	Genova
Moto	MT34567	M456	2024-07-08	Verona
Bici con rimorchio	BC12345	M012	2024-07-12	Milano
Furgone	FG9976	V776	2024-07-12	Palermo
Bici con rimorchio	BC34567	I334	2024-07-14	Napoli
Moto	MT45678	F876	2024-07-22	Bari
Furgone	FG12345	C432	2024-08-08	Torino
Furgone	FG23456	N123	2024-09-14	Bologna
Furgone	FG3317	[null]	[null]	[null]
Furgone	FG34567	[null]	[null]	[null]

Query 4 La seguente query permette la visualizzazione di tutti i destinatari di pacchi che hanno più di un tentativo di consegna

```sql
SELECT * FROM cliente c WHERE c.codice_cliente IN (
    SELECT c.codice_cliente FROM pacco p
    JOIN ordine o ON p.numero_ordine = o.numero_ordine
    JOIN cliente c ON c.codice_cliente = o.destinatario
    JOIN consegna co ON co.codice_tracciamento = p.codice_tracciamento
    WHERE co.esito = 'Tentata consegna'
    GROUP BY c.codice_cliente
    HAVING COUNT(*)>1
)
```

Estratto dell'output:

codice_ [PK] inte	nome character v	cognome character vary	telefono character v	email character varying (50)	riga_indirizzo1 character varying (50)	riga_indiriz character v	cap characte	citta character	paese charact
7	Antonio	Romano	357890...	[null]	Via Dante 7	[null]	30100	Venezia	Italia
41	Giovanni	Gentile	351345...	[null]	Via Tevere 45	Scala C	00100	Roma	Italia
43	Fabio	Marte	353567...	fabio.marte@email.com	Via Appia 23	[null]	80100	Napoli	Italia
46	Claudia	Parisi	356890...	claudia.parisi@email.com	Largo Argentina 78	Int. 2	90100	Palermo	Italia
47	Matteo	Neri	357901...	[null]	Corso Adriatico 12	[null]	60100	Ancona	Italia
51	Alberto	Ferri	351345...	alberto.ferri@email.com	Via Monte Bianco 10	[null]	10100	Torino	Italia
54	Diana	D'Amico	354678...	diana.damico@email.com	Corso Buenos Aires 25	[null]	20100	Milano	Italia
61	Leonar...	Rossetti	351345...	[null]	Viale delle Milizie 60	[null]	00100	Roma	Italia
55	Edoardo	Pace	355719...	[null]	Via Flaminia 30	[null]	00100	Roma	Italia
45	Luca	Orlando	355781...	luca.orlando@email.com	Viale dei Platani 56	[null]	90100	Palermo	Italia
48	Greta	Colombo	351012...	greta.colombo@email.com	Via Padova 34	[null]	35100	Padova	Italia
42	Roberta	Leone	352456...	roberta.leone@email.com	Corso Trieste 67	[null]	34100	Trieste	Italia
50	Federica	Testa	350234...	[null]	Viale Venezia 89	[null]	30100	Venezia	Italia
57	Gianluca	Sartori	357901...	gianluca.sartori@email.com	Via Zamboni 40	[null]	40100	Bologna	Italia
56	Flavia	Valentini	356890...	flavia.valentini@email.com	Viale Trastevere 35	[null]	00100	Roma	Italia
53	Carlo	Sorrentino	353567...	carlo.sorrentino@email.com	Via Partenope 20	Piano 3	80100	Napoli	Italia
60	Katia	Benedetti	350234...	katia.benedetti@email.com	Via XX Settembre 55	App. 6	16100	Genova	Italia

Query 5 La seguente query permette di vedere il nome, cognome e numero di consegne andate a buon fine del corriere con il maggior numero di pacchi consegnati con successo

```sql
SELECT d.nome, d.cognome, COUNT(*) FROM consegna c
    JOIN opera o ON c.codice_tracciamento = o.codice_tracciamento
    JOIN dipendente d ON o.codice_corriere = d.codice_corriere
    WHERE esito='Consegnato'
    GROUP BY o.codice_corriere, d.nome, d.cognome
    HAVING COUNT(*) = (
        SELECT MAX(totale) FROM (
        SELECT COUNT(*) as totale FROM consegna c
        JOIN opera o ON c.codice_tracciamento = o.codice_tracciamento
        WHERE esito='Consegnato'
        GROUP BY codice_corriere
        )
    )
```

Estratto dell'output:

nome character varying (50)	cognome character varying (50)	count bigint
Vasile	Dumitru	14

5.2 Creazione degli indici

Supponendo di voler ottimizzare la query 2 e, considerando le varie operazioni di supervisione che un operatore di un'azienda di spedizione deve fare sugli ordini fatti dai clienti, è utile predisporre un indice, sulla colonna *DATA_ CREAZIONE*, che permetta di fare controlli e riordinamenti più rapidi.

```
CREATE INDEX indice_ordinamento_ordini ON public.ordine USING btree (data_creazione);
```

Tale indice, nel caso della query in questione, è utile per il filtraggio in base all'anno.
Considerando invece la query 3, sapendo che si fanno un'operazione di *join* su *DATA_ UTILIZZO* e un'operazione di *order by* su *DATA_ UTILIZZO* e *CODICE_ CORRIERE*, è utile, tenendo a mente che solitamente le aziende di spedizioni contano molti corrieri e molti veicoli, includere un indice che mantenga la relazione *guida* ordinata.

```
CREATE INDEX indice_ordinamento_data_corriere
ON public.guida USING btree (data_utilizzo, codice_corriere);
```

Sapendo che la maggior parte delle query che verranno fatte saranno di aggiornamento o di inserimento all'interno delle tabelle *guida* e *ordine*, si è deciso di utilizzare un indice *Btree*, rispetto ad un indice *hash*. Questo per poter sfruttare la capacità dei Btree di essere più efficiente su operazioni di aggiornamento e le ricerche ad intervalli.

6 Applicazione Software

All'interno del file **Spedizioniere.c** è stata predisposta la connessione al database e un'interfaccia utente che permette di eseguire le query in maniera intuitiva.
Inizialmente viene mostrato in output il menù delle interrogazioni che posso essere eseguite:

0. Per terminare l'applicazione.
1. Numero di pacchi gestiti da un determinato magazzino raggruppati per tipologia e con annessa la media del peso.
2. Numero di pacchi di un ordine generato di una determinata città e in un determinato anno.
3. Tutti i veicoli a disposizione dell'azienda con la data, corriere e città di ultimo utilizzo.
4. Tutti i dati dei clienti per cui ci sono serviti più di un tentativo di consegna.
5. Nome, cognome e numero di consegne del corriere con più consegne di successo.

In base al numero del menù inserito, sarà data la possibilità all'utente di eseguire la query corrispondente.
Se la query è parametrica (1 e 2) verranno, ovviamente, chiesti gli input necessari all'interrogazione. Per aiutare l'utente a selezionare degli input validi all'interno del database, sono stati predisposti degli output, anch'essi query, d'aiuto. Ad esempio, nella query 2 vengono fornite le città ove è stato inviato almeno un ordine.
Ad ogni query eseguita viene richiamata la funzione *esegui_ query_ spaziato()* che, oltre a fare l'interrogazione, spazia i risultati delle query calcolando la dimensione di ogni colonna in base alla dimensione massima delle stringhe che ricadono sotto suddetta colonna, ottenendo un output leggibile. Inoltre avviene un controllo, mediante la funzione *checkResults()*, che stampa i possibili errori presenti sulle query.

Main del file Spedizioniere.c

```c
int main(int args, char* argsv[]){
    char conninfo[250]; sprintf (conninfo,"user=%s dbname=%s hostaddr=%s port=%d",PG_USER,PG_DB,PG_HOST,PG_PORT); PGconn *conn = PQconnectdb(conninfo);
    if( PQstatus(conn) == CONNECTION_OK ){ printf("****************Connessione riuscita****************\n"); }
    else { printf("Connessione non riuscita: ERRORE: %s\n",PQerrorMessage(conn)); }
    char query[512]; int input = -1;
    while (input != 0 && PQstatus(conn) == CONNECTION_OK )
    {   printf("MENU QUERY PROGETTO SPEDIZIONIERE\n"
        "Le interrogazioni al database spedizioniere sono le seguenti:\n"
        "0 - Per terminare l'applicazione\n"
        "1 - Numero di pacchi gestiti da un determinato magazzino ragruppati per tipologia e con annessa la media del peso\n"
        "2 - Numero di pacchi di un ordine generato di una determinata citta e in un determinato anno\n"
        "3 - Tutti i veicoli a disposizione dell'azienda con la data, corriere e città di utimo utilizzo\n"
        "4 - Tutti i dati dei clienti per cui ci sono serviti più di un tentativo di consegna\n"
        "5 - Nome, cognome e numero di consegne del corriere con più consegne di successo\n");
        printf("Prego, inserire l'interrogazione che si vuole eseguire: "); scanf("%d", &input);
        switch (input) {
            case 0: input=0; break;
            case 1:
                printf("\nIn seguito visualizzerai i vari magazzini disponibili\n");
                esegui_query_spaziato(conn,"SELECT codice_magazzino, citta, nome FROM magazzino");
                printf("\n\n"); int codice_magazzino;
                printf("Prego, inserire un codice di magazzino di quelli sovrastanti: ");
                scanf("%d", &codice_magazzino);
                snprintf(query, sizeof(query),"SELECT tipo_contenuto, COUNT(*) as quantita, ROUND(AVG(p.peso),2) "
                    "as media_peso FROM pacco p, magazzino m "
                    "WHERE p.codice_magazzino = m.codice_magazzino "
                    "AND p.codice_magazzino = %d "
                    "AND ( p.stato='In magazzino' OR p.stato='In attesa') "
                    "GROUP BY tipo_contenuto "
                    "ORDER BY tipo_contenuto", codice_magazzino);
                esegui_query_spaziato(conn,query); break;
            case 2:
                printf("\nIn seguito vedrai tutte le città dalle quali è possibile utilizzare i nostri servizi\n");
                esegui_query_spaziato(conn,"SELECT DISTINCT citta FROM ordine o JOIN cliente c ON o.mittente = c.codice_cliente"); printf("\n\n\n");
                char citta[50]; int anno; printf("Città: "); scanf("%s", citta); printf("Anno: "); scanf("%d", &anno);
                snprintf(query, sizeof(query),"SELECT COUNT(*) FROM pacco p "
                    "JOIN ordine o ON p.numero_ordine = o.numero_ordine "
                    "JOIN cliente c ON c.codice_cliente = o.mittente "
                    "WHERE c.citta='%s' AND EXTRACT(YEAR FROM o.data_creazione) = %d", citta, anno);
                esegui_query_spaziato(conn,query); break;
            case 3:
                esegui_query_spaziato(conn,"SELECT v.tipo, r.targa, g.codice_corriere, r.data_ultimo_utilizzo, m.citta "
                    "FROM guida g RIGHT JOIN ( "
                        "SELECT v.targa, MAX(data_utilizzo) AS data_ultimo_utilizzo FROM veicolo v "
                        "LEFT JOIN guida g ON g.targa = v.targa "
                        "GROUP BY v.targa ) r "
                    "ON g.targa = r.targa AND g.data_utilizzo = r.data_ultimo_utilizzo "
                    "LEFT JOIN veicolo v ON v.targa = r.targa "
                    "LEFT JOIN dipendente d ON d.codice_corriere = g.codice_corriere "
                    "LEFT JOIN magazzino m ON m.codice_magazzino = d.codice_magazzino "
                    "ORDER BY data_ultimo_utilizzo, g.codice_corriere;"); break;
            case 4:
                esegui_query_spaziato(conn,"SELECT * FROM cliente c WHERE c.codice_cliente IN "
                    "(SELECT c.codice_cliente FROM pacco p "
                    "JOIN ordine o ON p.numero_ordine = o.numero_ordine "
                    "JOIN cliente c ON c.codice_cliente = o.destinatario "
                    "JOIN consegna co ON co.codice_tracciamento = p.codice_tracciamento "
                    "WHERE co.esito = 'Tentata consegna' "
                    "GROUP BY c.codice_cliente "
                    "HAVING COUNT(*)>1)"); break;
            case 5:
                esegui_query_spaziato(conn,"SELECT d.nome, d.cognome, COUNT(*) FROM consegna c "
                    "JOIN opera o ON c.codice_tracciamento = o.codice_tracciamento "
                    "JOIN dipendente d ON o.codice_corriere = d.codice_corriere "
                    "WHERE esito='Consegnato' "
                    "GROUP BY o.codice_corriere, d.nome, d.cognome "
                    "HAVING COUNT(*) = ( "
                        "SELECT MAX(totale) FROM ( "
                            "SELECT COUNT(*) as totale FROM consegna c "
                            "JOIN opera o ON c.codice_tracciamento = o.codice_tracciamento "
                            "WHERE esito='Consegnato' "
                            "GROUP BY codice_corriere ))"); break;
            default: printf("Valore non valido. Inserisci un numero da 0 a 5.\n"); break; } }
    PQfinish(conn); return 0;
}
```